HKT IoT Platform Open API

User Guide

Document Version: V5.7
Release Date: 2026-05-29

1. Quick Start

1.1 Document Overview

This document aims to help developers quickly get started with HKT IoT Platform Open API, completing device integration, space management, and device control functions.

Target Audience:

Integration Development Engineers

Technical Leads

Partner Technical Teams

1.2 Environment Preparation

Before starting to use the API, please complete the following preparations:

1.2.1 Obtain Configuration Information

Please contact your sales representative or project manager to obtain the following configuration information:

1. HKT LoRaWAN Network-Service Configuration

oServer IP address and port number

2. HKT IoT Platform Open API Configuration

oBase-URL (e.g., https://<host>/)

3. Application Credentials

oApplication ID (appId)

oApplication Secret (appSecret)

1.2.2 Network Configuration Check

Please ensure the following network configurations are completed:

LoRaWAN Network Configuration: Ensure your LoRaWAN gateway can access the HKT LoRaWAN Network-Service server, configure firewall rules if necessary.

Open API Network Configuration: Ensure your network can access the HKT IoT Platform Open API Base-URL, configure firewall rules if necessary.

1.2.3 Document Preparation

Please ensure you have read and understood the "HKT IoT Platform Open API — Customer Integration Guide" document.

⚠ Security Reminder:

appId and appSecret are only used for creating and managing API-KEYs

Do not use these two credentials in daily business API calls

Do not log appSecret or commit it to code repositories

Please securely store all configuration information

1.3 Quick Start Guide

This chapter will guide you to complete your first API call within 5 minutes.

Step 1: Create API-KEY

First, use your appId and appSecret to create an API-KEY:

POST /v1/api-keys
Authorization: Basic <base64(appId + ":" + appSecret)>
Content-Type: application/json

```
{
 "scope": "read_write",
 "description": "Quick start test",
 "expires_in_days": 30
}
```

Response Example (201 Created):

```
{
 "key_id": "12345",
 "api_key": "ak_live_abc123xyz456...",
 "description": "Quick start test",
```

"scope": "read_write",
"expires_at": "2026-06-28T12:00:00Z",
"created_at": "2026-05-29T12:00:00Z"
}

⚠ **Important:** api_key will only be displayed once during creation, please save it securely immediately!

Step 2: Call Business API

Use the API-KEY just created to query the space list:

```
GET /v1/spaces?page=1&pageSize=10
X-API-Key: ak_live_abc123xyz456...
```

Step 3: View Response

You will receive a response similar to the following:

{
"data": [],
"total": 0,
"page": 1,
"pageSize": 10
}

Congratulations! You have successfully completed your first API call. Please read the following chapters to learn more features.

---

2. API Fundamentals

2.1 Overall Call Flow

The complete Open API call flow is as follows:

1. Obtain application credentials (appId + appSecret)

↓

2. Use appId + appSecret to create API-KEY

↓

3. Use API-KEY to call business APIs (space/device/command, etc.)

↓

4. Process API response

Detailed Step Description:

1. **Preparation Phase**: Complete environment preparation, obtain appId and appSecret

2. **Authentication Phase**: Use appId and appSecret via HTTP Basic Auth to call the /v1/api-keys interface to create API-KEY

3. **Business Call Phase**: Use API-KEY (via X-API-Key or Authorization: Bearer request headers) to call various business APIs

4. **Result Processing Phase**: Parse API responses, process business data or error information

---

2.2 API Request Format

2.2.1 Request Basic Information

Base URL: {HKT_IoT_Platform_Open_API_Base_URL}

Protocol: HTTPS

Character Encoding: UTF-8

Content-Type: application/json (request body)

2.2.2 Common Request Headers

Header	Description	Example
Content-Type	Request body type, fixed as application/json	application/json
X-API-Key	API-KEY (used for business API calls)	ak_live_abc123xyz456...
Authorization	Authentication method (choose one): - Basic Auth (for API-KEY management) - Bearer Token (for business	Basic YXBwSWQ6YXBwU2VjcmV0 Bearer ak_live_abc123xyz456...

Header	Description	Example
	API calls)	
X-Trace-Id	Trace ID, used for troubleshooting (optional)	trace-123456

2.2.3 Request Methods

Method	Description
GET	Query resources
POST	Create resources or send commands
PUT	Update resources
DELETE	Delete resources (requires admin permission)

2.2.4 ID and Parameter Specifications

Numeric IDs: External IDs (device_id, space_id, parent_id, `space_Id` in device list queries, etc.) are all **decimal numeric strings**, **1-21 characters** (no letters).

Common Query Parameters:

Parameter	Type	Required	Description	Example
page	Integer	No	Page number, default 1	page=1
pageSize	Integer	No	Items per page, default 20, maximum 200	pageSize=20

Optional Filters: Unneeded query parameters can be omitted.

2.3 Permissions and HTTP Methods

scope	Allowed HTTP Methods
read	GET
write	POST, PUT
read_write	GET, POST, PUT

scope	Allowed HTTP Methods
admin	GET, POST, PUT, DELETE

Important Note: Deleting devices and spaces requires admin permission. Returns **403 Forbidden** when permissions are insufficient.

2.4 API Response Format

2.4.1 Success Response

List Query Response (pagination, using space list as example):


```
{
 "data": [
  {
   "space_id": "1",
   "name": "Example Space",
   "parent_id": null,
   "root_id": "1",
   "created_at": "2026-05-29T12:00:00Z"
  }
 ],
 "total": 100,
 "page": 1,
 "pageSize": 20
}
```


Single Resource Response (using space details as example):


```
{
 "space_id": "1",
 "name": "Example Space",
 "parent_id": null,
 "root_id": "1",
 "created_at": "2026-05-29T12:00:00Z"
}
```

Creation Success Response (using space creation as example):


```
{
 "space_id": "2",
 "name": "Newly Created Space",
 "parent_id": "1",
 "root_id": "1",
 "created_at": "2026-05-29T12:00:00Z"
}
```


2.4.2 Error Response


```
{
 "error": "ERROR_CODE",
 "details": "Error detailed description",
 "request_id": "trace-123456"
}
```


Common Error Codes:

HTTP Status Code	Error Code	Description
400	INVALID_REQUEST	Request parameter error
400	INVALID_SN	Invalid device serial number
400	INVALID_SPACE	Invalid space
401	UNAUTHORIZED	Unauthorized or authentication failed
401	KEY_EXPIRED	API-KEY expired
403	FORBIDDEN	Insufficient permissions
404	NOT_FOUND	Resource not found

3. API-KEY Management

3.1 Create API-KEY

Use appId and appSecret to create API-KEY:

POST /v1/api-keys

Authorization: Basic <base64(appId + ":" + appSecret)>

Content-Type: application/json

```
{
 "scope": "read_write",
 "description": "Smart Building Application API-KEY",
 "expires_in_days": 365
}
```

Request Parameter Description:

Parameter	Required	Description
scope	Yes	Permission scope: read, write, read_write, admin
description	No	Description, maximum 100 characters
expires_in_days	No	Validity period (1-3650 days), permanent if not specified

Response Example:

```
{
 "key_id": "12345",
 "api_key": "ak_live_abc123xyz456...",
 "description": "Smart Building Application API-KEY",
 "scope": "read_write",
 "expires_at": "2027-05-29T12:00:00Z",
 "created_at": "2026-05-29T12:00:00Z"
}
```

⚠ **Important:** api_key will only be displayed once during creation, please save it securely immediately!

3.2 Use API-KEY

Except for the /v1/api-keys path, all other API calls use API-KEY for authentication, with two methods:

Method 1: Use X-API-Key Request Header

```
GET /v1/spaces
X-API-Key: ak_live_abc123xyz456...
```

Method 2: Use Authorization Bearer Request Header

```
GET /v1/spaces
Authorization: Bearer ak_live_abc123xyz456...
```

3.3 View API-KEY List

```
GET /v1/api-keys?page=1&pageSize=20
Authorization: Basic <base64(appId + ":" + appSecret)>
```

Response Example:


```
{
 "data": [
  {
   "key_id": "12345",
   "description": "Smart Building Application API-KEY",
   "scope": "read_write",
   "status": "active",
   "expires_at": "2027-05-29T12:00:00Z",
   "last_used_at": "2026-05-29T12:30:00Z",
   "created_at": "2026-05-29T12:00:00Z"
  }
 ],
 "total": 1,
 "page": 1,
 "pageSize": 20
}
```


3.4 Rotate API-KEY

Regularly rotating API-KEYs is good security practice:

PUT /v1/api-keys/{key_id}/rotate
Authorization: Basic <base64(appId + ":" + appSecret)>

The response will return a new api_key.

3.5 Revoke API-KEY

If an API-KEY is compromised or no longer needed, revoke it immediately:

DELETE /v1/api-keys/{key_id}
Authorization: Basic <base64(appId + ":" + appSecret)>

---

4. Space Management

Spaces represent device installation locations, organized in a tree structure.

4.1 Create Root Space

POST /v1/spaces
X-API-Key: <your_api_key>
Content-Type: application/json

```
{
 "name": "Warsaw Headquarters Building"
}
```

4.2 Create Child Space

POST /v1/spaces
X-API-Key: <your_api_key>
Content-Type: application/json

```
{
 "name": "3rd Floor Data Center",
 "parent_id": "1"
}
```

Description: parent_id is the parent space ID, if not provided, creates a root space.

4.3 Query Space List

GET /v1/spaces?page=1&pageSize=20
X-API-Key: <your_api_key>

Supported query parameters:

name: Filter by name

parent_id: Filter by parent space ID (query child spaces of a space)

4.4 Query Single Space Details

GET /v1/spaces/{space_id}
X-API-Key: <your_api_key>

Response Example:

```
{
 "space_id": "1",
 "name": "Warsaw Headquarters Building",
 "parent_id": null,
 "root_id": "1",
 "created_at": "2026-05-29T12:00:00Z"
}
```

4.5 Update Space

PUT /v1/spaces/{space_id}
X-API-Key: <your_api_key>
Content-Type: application/json

```
{
 "name": "Warsaw Headquarters Building (New Name)",
 "parent_id": "2"
}
```

4.6 Delete Space

⚠ Requires admin permission:

DELETE /v1/spaces/{space_id}
X-API-Key: <your_api_key>

5. Device Management

5.1 Register Device

POST /v1/devices

X-API-Key: <your_api_key>

Content-Type: application/json

```
{
 "sn": "SN123456789",
 "name": "Temperature Humidity Sensor-001",
 "space_Id": "2"
}
```

Request Parameter Description:

Parameter	Required	Description
sn	Yes	Device serial number/license, maximum 100 characters
name	No	Device display name
space_Id	No	Bound space ID

5.2 Query Device List

GET /v1/devices?page=1&pageSize=20

X -API-Key: <your_api_key>

Supported query parameters:

keyword: Search devices by keyword

space_Id: Filter by space ID (query devices in a space)

Example: Query devices in a space

GET /v1/devices?space_Id=2&page=1&pageSize=20

XI -API-Key: <your_api_key>

5.3 Query Device Details

GET /v1/devices/{device_id}

X-API-Key: <your_api_key>

Response Example:

{
"device_id": "1001",
"name": "Temperature Humidity Sensor-001",
"type": "temperature_humidity",
"type_name": "Temperature Humidity Sensor",
"status": "online",
"status_code": 1,
"created_at": "2026-05-29T12:00:00Z",
"last_active_at": "2026-05-29T14:30:00Z",
"identifier": "device-001",
"rssi": -65,
"snr": 25
}

5.4 Update Device

PUT /v1/devices/{device_id}

X-API-Key: <your_api_key>

Content-Type: application/json

{
"name": "Temperature Humidity Sensor-001 (New Name)"
}

5.5 Delete Device

⚠ Requires admin permission:

DELETE /v1/devices/{device_id}

X-API-Key: <your_api_key>

---

6. Device Commands

6.1 Send Device Command

POST /v1/devices/{device_id}/commands

X -API-Key: <your_api_key>

Content-Type: application/json

```
{
"func": {
  "method": "YOUR_FUNCTION_NAME"
 }
}
```

Description:

device_id: Target device ID (decimal numeric string, 1-21 characters)

func.method: Device function/command name (specific parameter settings refer to "6.3 Device Command Reference" chapter)

func.params: Optional, command parameters (JSON object, depends on function)

Command Example with Parameters (using door lock RemoteOpenClose as example):

POST /v1/devices/{device_id}/commands

XI -API-Key: <your_api_key>

Content-Type: application/json

```
{
"func": {
"method": "RemoteOpenClose",
"params": {
  "lockControl": 1
 }
}
```

}

Response Example:

{
"total_count": 1,
"success_count": 1,
"fail_count": 0,
"success_list": [
{
"record_id": "54321",
"device_id": "1001"
}
],
"fail_list": []
}

Important: Please save success_list[].record_id for subsequent command status queries.

6.2 Query Command Status

Use the record_id returned when sending the command to query execution status:

GET /v1/commands/{command_id}/status
X-API-Key: <your_api_key>

Path Parameters:

command_id: The previously returned record_id (decimal numeric string, 1-21 characters)

Response Example (using snake_case):

{
"record_id": "54321",
"device_id": "1001",
"func_name": "RemoteOpenClose",
"func_params": "{\"lockControl\":1}",
"cmd_state": 1,
"cmd_state_text": "Success",
"error_msg": **null**,

"create_time": "2026-05-29T14:00:00Z",
"operator_name": "API Call"
}

Response Field Detailed Description:

Field	Type	Description
record_id	string	Record ID
device_id	string	Device ID
func_name	string	Command name
func_params	string	Command parameters
cmd_state	integer	Status code (see table below)
cmd_state_text	string	Status description
error_msg	string	Failure reason
create_time	string	Operation time (datetime)
operator_name	string	Operator display name

Command Status Code Description:

Status Code	Description
-1	Pending Send
0	Sending
1	Success
2	Failed
3	Timeout
4	Pending Retry
5	Expired

6.3 Device Command Reference

6.3.1 Door Lock Command List

1. RemoteOpenClose — Remote Lock Open/Close (0x2F)

Function: Remotely control door lock opening or closing.

JSON Example:

```
{
 "func": {
  "method": "VolumeSetting",
  "params": {
   "volume": 80
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
volume	int	Volume percentage	Range 0~100

---

7. RestoreDefaultFactorySettings — Restore Factory Default Settings (0x85)

Function: Restore device to factory default state (only available when not bound).

JSON Example:

```
{
 "func": {
  "method": "RestoreDefaultFactorySettings",
  "params": {}
 }
}
```

Parameter Description: No parameters.

---

8. DataSyncPeriodSetting — Data Synchronization Period (0x86)

Function: Set device active status synchronization interval to server.

JSON Example:

```
{
 "func": {
  "method": "DataSyncPeriodSetting",
  "params": {
   "dataSyncPeriod": 1440
```

```
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
dataSyncPeriod	int	Data synchronization period (minutes)	Range 10~1440, 0=No active synchronization

---

9. TimezoneSetting — Timezone Setting (0x8A)

Function: Set device timezone.

JSON Example:

```
{
 "func": {
  "method": "TimezoneSetting",
  "params": {
   "timeZone": 8
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
timeZone	int	0~12=E0~E12 zones, 13~24=W1~W12 zones, 25=UTC+3.5, 26=UTC+5.5	Range 0~26

Timezone Reference Table:

Value	Meaning	Value	Meaning
0	UTC+0	13	UTC-1
8	UTC+8 (East 8 Zone)	17	UTC-11
25	UTC+3.5	26	UTC+5.5

6.3.2 Other Device Command Lists

Gas Meter

1. Set Gas Unit Price (gasUnitPrice)

```
{
 "func": {
  "method": "gasUnitPrice",
  "params": {
   "gasUnitPrice": 3.5
  }
 }
}
```

2. Gas Recharge (gasCharge)

```
{
 "func": {
  "method": "gasCharge",
  "params": {
   "gasBalance": 200
  }
 }
}
```

3. Set Gas Usage (gasUsage)

```
{
 "func": {
  "method": "gasUsage",
  "params": {
   "gasUsage": 50.5
  }
 }
}
```

4. Set Gas Surplus (gasSurplus)

```
{
 "func": {
```

```
  "method": "gasSurplus",
  "params": {
   "gasSurplus": 150.5
  }
 }
}
```

5. Set Gas Balance (gasBalance)

```
      {
 "func": {
  "method": "gasBalance",
  "params": {
   "gasBalance": 200
  }
 }
}
```

6. Valve Control (valveControl)

```
      {
 "func": {
  "method": "valveControl",
  "params": {
   "valveStatus": 0
  }
 }
}
```

valveStatus	Description
0	Close valve
1	Open valve

Heating Control Valve

1. Set Target Temperature (SetTargetTemperature)

```
      {
 "func": {
```

```
  "method": "SetTargetTemperature",
  "params": {
   "targetTemperature": 25.5,
   "temperatureTolerance": 0.5
  }
 }
}
```

2. Set Valve Opening (SetValveOpening)

```
    {
 "func": {
  "method": "SetValveOpening",
  "params": {
   "valveOpening": 50
  }
 }
}
```

valveOpening	Description
0-100	Valve opening percentage

Parking Lock

1. Lock Control (LockControl)

```
    {
 "func": {
  "method": "LockControl",
  "params": {
   "lockStatus": 1,
   "bluetoothId": "BT0012345678"
  }
 }
}
```

lockStatus	Description
1	Lower lock

lockStatus	Description
2	Raise lock
3	APP lower lock
4	APP raise lock

Switch Panel

1. Switch Control (SwitchControl)

```
{
 "func": {
  "method": "SwitchControl",
  "params": {
   "state": 1
  }
 }
}
```

state	Description
0	Off
1	On

Sound and Light Alarm

1. Alarm Control (AlarmControl)

```
{
 "func": {
  "method": "AlarmControl",
  "params": {
   "alarm": 1,
   "alarmTime": 10
  }
 }
}
```

alarm	Description
0	Off
1	On

alarmTime	Description
Value	Alarm duration (seconds)

Air Switch Thing Model

1. Air Switch Control (CtrlAirSwitchOnOff)

```
    {
 "func": {
  "method": "CtrlAirSwitchOnOff",
  "params": {
   "onOff": 1
  }
 }
}
```

onOff	Description
0	Off (close circuit)
1	On (open circuit)

Electric Meter

1. Valve Control (ValveControl)

```
    {
 "func": {
  "method": "ValveControl",
  "params": {
   "valveStatus": 0
  }
```

```
 }
}
```

valveStatus	Description
0	Open valve
1	Close valve

2. Remote Recharge (RemoteRecharge)

```
    {
 "func": {
  "method": "RemoteRecharge",
  "params": {
   "amount": 100.00
  }
 }
}
```

Water Meter

1. Valve Control (ValveControl)

```
    {
 "func": {
  "method": "ValveControl",
  "params": {
   "valveStatus": 0,
   "meterAddress": "ADDR1234"
  }
 }
}
```

valveStatus	Description
0	Open valve
1	Close valve

Ultrasonic Water Meter

1. Valve Control (ValveControl)

```
    {
 "func": {
  "method": "ValveControl",
  "params": {
   "valveStatus": 0
  }
 }
}
```

valveStatus	Description
0	Open valve
1	Close valve
2	Unclog

2. Set Metering Mode (SetMeteringMode)

```
    {
 "func": {
  "method": "SetMeteringMode",
  "params": {
   "meteringMode": 0
  }
 }
}
```

meteringMode	Description
0	Dual pulse
1	Single pulse
2	Hall
3	ADC collection
4	Photoelectric direct reading

3. Set Pulse Constant (SetPulseConstant)

```
    {
 "func": {
```

```
  "method": "SetPulseConstant",
  "params": {
   "pulseConstant": 1
  }
 }
}
```

pulseConstant	Description
1	1 metering pulse represents 1 liter
2	1 metering pulse represents 10 liters
3	1 metering pulse represents 100 liters
4	1 metering pulse represents 1000 liters

7. Security Recommendations and FAQs

7.1 Security Recommendations

Do not log complete API Keys or appSecret in logs.

Store API Keys or appSecret in secure key management systems (such as Vault).

Regularly rotate Keys (PUT /v1/api-keys/{key_id}/rotate), revoke immediately if compromised.

7.2 Frequently Asked Questions

Q: How to obtain the function name list corresponding to device types?

A: Please refer to the 6.3 Device Command Reference chapter in this document, which includes complete command lists for door locks, gas meters, heating control valves, parking locks, switch panels, sound and light alarms, air switch thing models, electric meters, water meters, ultrasonic water meters, and other devices. For more device type functions, please contact the HKT project team.

Q: How long after sending a command can status be queried?

A: It is recommended to wait 2-5 seconds after sending the command before querying status, depending on device type and network conditions. Use the returned record_id to call GET /v1/commands/{record_id}/status to query execution status.

Q: How to send device commands?

A: Use the POST /v1/devices/{device_id}/commands interface, specify the function name in func.method in the request body, and provide parameters in func.params. For details, refer to the detailed examples in 6.3 Device Command Reference.

Q: What to do when API call returns 403 Forbidden?

A: Please check if your API-KEY permission scope is sufficient, delete operations require admin permission. Sending device commands usually requires write or read_write permission.

Q: Can spaces be nested infinitely?

A: API supports parent-child relationship nesting, but please follow reasonable hierarchical structure design.

Q: Can a device belong to multiple spaces simultaneously?

A: No, a device can only be bound to one space at a time.

---

8. Appendix: Complete Integration Process Example

The following are complete integration steps for smart building application scenarios:

1. Receive Credentials: Obtain appId and appSecret through secure channels

2. Create API-KEY:

POST /v1/api-keys

```
{
 "scope": "read_write",
 "description": "Smart Building Application Project Integration",
 "expires_in_days": 365
}
```

3. Create Space Tree:

oCreate root space: POST /v1/spaces → {"name": "Poland Region"}

oCreate city space: POST /v1/spaces → {"name": "Warsaw", "parent_id": "1"}

oCreate building space: POST /v1/spaces → {"name": "Headquarters Building", "parent_id": "2"}

4. Register Device:

POST /v1/devices

```
{
 "sn": "SN-PL-001",
 "name": "Warsaw-Headquarters-001",
 "space_Id": "3"
}
```

5. Query Devices in Space:

GET /v1/devices?space_Id=3

6. Send Device Commands:

oSend command to obtain record_id

oPoll GET /v1/commands/{record_id}/status to query status when needed

---

9. Document Notes and Specifications

This document maintains consistency with the agreed customer-side space model:

No exposure of hierarchical API

Create space uses name + optional parent_id (if omitted, creates top-level space)

Field names in JSON use **snake_case** in defined places (device_id, space_id, parent_id, expires_in_days, etc.); space_Id in device registration/list queries uses camelCase

---

10. Technical Support

If you encounter problems, please contact your sales representative or project manager, and provide the following information:

Environment URL

Requested API path and method

request_id or X-Trace-Id (if available)

Error information and request time

"method": "RemoteOpenClose",
"params": {
"lockControl": 1
}
}
}

Parameter Description:

Parameter	Type	Description	Constraints
lockControl	int	0=Remote unlock, 1=Remote lock	Required, only allows 0 or 1

---

2. ManageTmpPwd — Temporary Password (0x32)

Function: Issue temporary unlock password with validity period.

JSON Example:

```json
{
 "func": {
  "method": "ManageTmpPwd",
  "params": {
   "validDuration": 30,
   "pwd": "123456"
  }
```

```
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
validDuration	int	Validity period (minutes, maximum 24 hours)	Required, range 1~1440
pwd	string	Temporary password	Required, length 6~8 digits

---

3. ManagePwd — Manage Password (with validity period) (0x4E)

Function: Manage user passwords (add/modify, delete, read), supports validity period and usage count limits.

JSON Example:

Add/Modify (operation = 0)

```
    {
 "func": {
  "method": "ManagePwd",
  "params": {
   "operation": 0,
   "userNo": 1,
   "pwdStatus": 1,
   "pwd": "12345678",
   "validStartTime": 1704038400,
   "validEndTime": 1706716800,
   "validUnlockCount": 0
  }
 }
}
```

Delete (operation = 1)

```
    {
 "func": {
```

```
  "method": "ManagePwd",
  "params": {
   "operation": 1,
   "userNo": 1
  }
 }
}
```

Read (operation = 2)

```
{
 "func": {
  "method": "ManagePwd",
  "params": {
   "operation": 2,
   "userNo": 1
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
operation	int	0=Add/Modify, 1=Delete, 2=Read	Required, 0/1/2
userNo	int	User number	Required, 0=Super admin, 1~100=Regular user, 101~200=Bluetooth user
pwdStatus	int	Password status (required when adding)	0=Frozen, 1=Active
pwd	string	Password (required when adding)	Length 6~8 digits
validStartTime	int	Valid start time (Unix seconds,	Required

Parameter	Type	Description	Constraints
		GMT)	
validEndTime	int	Valid end time (Unix seconds, GMT)	Required
validUnlockCount	int	Valid unlock count	0=No limit, >0=Limited count

4. ManageCard — Manage MF Card (with validity period) (0x4F)

Function: Manage IC cards/access cards, supports validity period and usage count limits.

JSON Example:

Add/Modify (operation = 0)

```
    {
 "func": {
  "method": "ManageCard",
  "params": {
   "operation": 0,
   "userNo": 1,
   "cardStatus": 1,
   "cardNo": "01020304",
   "cardKey": "0102030405060708010203040506070 8",
   "validStartTime": 1704038400,
   "validEndTime": 1706716800,
   "validUnlockCount": 0
  }
 }
}
```

Delete (operation = 1)

```
    {
 "func": {
  "method": "ManageCard",
```

```
  "params": {
    "operation": 1,
    "userNo": 1
  }
 }
}
```

Read (operation = 2)

```
     {
 "func": {
  "method": "ManageCard",
  "params": {
    "operation": 2,
    "userNo": 1
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
operation	int	0=Add/Modify, 1=Delete, 2=Read	Required
userNo	int	User number	1~100
cardStatus	int	Card status (required when adding)	0=Frozen, 1=Active
cardNo	string	Card number (hexadecimal string, no 0x)	8 hexadecimal characters (4 bytes)
cardKey	string	Card key (hexadecimal string)	32 hexadecimal characters (16 bytes)
validStartTime	int	Valid start timestamp	Required

Parameter	Type	Description	Constraints
validEndTime	int	Valid end timestamp	Required
validUnlockCount	int	Valid unlock count	0=Unlimited

5. NormallyOpenModeSetting — Normally Open Mode Setting (0x52)

Function: Set door lock normally open mode, supports manual locking, delayed locking, and timed normally open sub-modes.

JSON Example:

Mode 0: Disable Normally Open Mode

```
{
 "func": {
  "method": "NormallyOpenModeSetting",
  "params": {
   "mode": 0
  }
 }
}
```

Mode 1: Normally Open Mode 1 (Manual Locking, press 35# to lock)

```
{
 "func": {
  "method": "NormallyOpenModeSetting",
  "params": {
   "mode": 1
  }
 }
}
```

Mode 2: Normally Open Mode 2 (Delayed Auto Locking)

```
{
 "func": {
```

```
  "method": "NormallyOpenModeSetting",
  "params": {
   "mode": 2,
   "delayLockTime": 1800
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
mode	int	0=Disable, 1=Mode 1, 2=Mode 2	Required
delayLockTime	int	Delayed lock time (seconds, required for mode 2)	Range 1~65535

---

6. LockBackTimeSetting — Door Lock Auto Relock Time (0x53)

Function: Set delayed auto relock time after unlocking.

JSON Example:

```
{
 "func": {
  "method": "LockBackTimeSetting",
  "params": {
   "lockBackTime": 5
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
lockBackTime	int	Auto relock time (seconds)	Range 3~30

---

7. SyncTimestamp — Synchronize Timestamp (0x54)

Function: Synchronize device local time with server time, or read device current time.

JSON Example:

```
{
 "func": {
  "method": "SyncTimestamp",
  "params": {
   "timestamp": 1704038400
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
timestamp	int	Unix timestamp (seconds, GMT)	Timestamp

---

8. UserBindingStatusSetting — User Binding Status (0x56)

Function: Set device binding status with platform.

JSON Example:

```
{
 "func": {
  "method": "UserBindingStatusSetting",
  "params": {
   "userBindingStatus": 1
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
userBindingStatus	int	0=Not bound, 1=Bound	Required, only allows 0 or 1

9. VolumeSetting — Volume Setting (0x57)

Function: Set device volume percentage.

JSON Example:

```
{
 "func": {
  "method": "VolumeSetting",
  "params": {
   "volume": 80
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
volume	int	Volume percentage	Range 0~100

10. RestoreDefaultFactorySettings — Restore Factory Settings (0x85)

Function: Restore device to factory default state (only available when device is not bound).

JSON Example:

```
{
 "func": {
  "method": "RestoreDefaultFactorySettings",
  "params": {}
 }
}
```

Parameter Description: No parameters required.

11. DataSyncPeriodSetting — Data Sync Period Setting (0x86)

Function: Set device data synchronization period.

JSON Example:

```
{
 "func": {
  "method": "DataSyncPeriodSetting",
  "params": {
   "syncPeriod": 3600
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
syncPeriod	int	Sync period (seconds)	Range 60~86400

---

12. TimezoneSetting — Timezone Setting (0x8A)

Function: Set device timezone.

JSON Example:

```
{
 "func": {
  "method": "TimezoneSetting",
  "params": {
   "timezone": "Asia/Shanghai"
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
timezone	string	Timezone identifier	IANA timezone format

6.3.3 Other Device Command List

Gas Meter

1. Set Gas Unit Price (gasUnitPrice)

Function: Set gas unit price.

JSON Example:

```
{
 "func": {
  "method": "gasUnitPrice",
  "params": {
   "unitPrice": 3.5
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
unitPrice	float	Gas unit price	Positive number

2. Gas Recharge (gasCharge)

Function: Recharge gas.

JSON Example:

```
{
 "func": {
  "method": "gasCharge",
  "params": {
   "amount": 100.0
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
amount	float	Recharge amount	Positive number

3. Set Gas Usage (gasUsage)

Function: Set gas usage.

JSON Example:

```
{
 "func": {
  "method": "gasUsage",
  "params": {
   "usage": 50.0
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
usage	float	Gas usage	Positive number

4. Set Gas Surplus (gasSurplus)

Function: Set gas surplus.

JSON Example:

```
{
 "func": {
  "method": "gasSurplus",
  "params": {
   "surplus": 200.0
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
surplus	float	Gas surplus	Positive number

5. Set Gas Balance (gasBalance)

Function: Set gas balance.

JSON Example:

```
    {
 "func": {
  "method": "gasBalance",
  "params": {
   "balance": 150.0
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
balance	float	Gas balance	Positive number

6. Valve Control (valveControl)

Function: Control gas valve.

JSON Example:

```
    {
 "func": {
  "method": "valveControl",
  "params": {
   "state": 1
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
state	int	Valve state: 0=Close, 1=Open	Required

Heating Control Valve

1. Set Target Temperature (SetTargetTemperature)

Function: Set target temperature.

JSON Example:

```
    {
 "func": {
  "method": "SetTargetTemperature",
  "params": {
   "temperature": 22.5
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
temperature	float	Target temperature	Range -40~85

2. Set Valve Opening (SetValveOpening)

Function: Set valve opening percentage.

JSON Example:

```
    {
 "func": {
  "method": "SetValveOpening",
  "params": {
   "opening": 75
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
opening	int	Valve opening percentage	Range 0~100

Parking Lock

1. Lock Control (LockControl)

Function: Control parking lock.

JSON Example:

```
    {
 "func": {
  "method": "LockControl",
  "params": {
   "action": "lock"
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
action	string	Action: "lock" or "unlock"	Required

Switch Panel

1. Switch Control (SwitchControl)

Function: Control switch.

JSON Example:

```
    {
 "func": {
  "method": "SwitchControl",
  "params": {
   "channel": 1,
   "state": 1
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
channel	int	Switch channel	Required
state	int	Switch state: 0=Off, 1=On	Required

Sound and Light Alarm

1. Alarm Control (AlarmControl)

Function: Control alarm.

JSON Example:

```
{
 "func": {
  "method": "AlarmControl",
  "params": {
   "state": 1
  }
 }
}
```

Parameter Description:

Parameter	Type	Description	Constraints
state	int	Alarm state: 0=Off, 1=On	Required

---

7. Document Specifications and Conventions

This document is consistent with the agreed customer-side space model:

No hierarchical API exposure

Create space uses name + optional parent_id (if omitted, creates top-level space)

Field names in JSON use **snake_case** (device_id, space_id, parent_id, expires_in_days, etc.) at defined locations; `space_Id` in device registration/list queries uses camelCase

8. Technical Support

If you encounter issues, please contact your sales representative or project manager, and provide the following information:

Environment URL

Requested API path and method

request_id or X-Trace-Id (if available)

Error message and request time